AWS VPC PEERING

Secure Private Connectivity Between Two VPCs

Aws Cloud Networking Project Report

Submitted by: Shaik Shareef

Batch: September 2026

Project Title: AWS VPC Peering and Secure Communication Between EC2 Instances.

Cloud Platform: Amazon Web Services (AWS)

AWS Region: Asia Pacific (Sydney) / ap-southeast-2

Completion date: 17-10-2026

Project Workbook: EWB Cloud Labs – AWS VPC Perring

Project Summary:

This project focuses on connecting two Amazon Virtual Private Clouds (VPCs) through VPC peering. It demonstrates how to configure VPC networks, subnets, EC2 instances, route tables, and security groups to enable private communication between application instances.

The project also explores secure administrative access through a bastion host, connectivity verification using ping and SSH, and troubleshooting network communication issues.

Project Area: Cloud Computing and Networking

Key Technologies: Amazon VPC, VPC Peering, Amazon EC2, Route Tables, Security Groups, SSH, and ICMP

1. Project Overview

1.1 Introduction

Amazon Web Services (AWS) provides cloud computing services that allow users to create, configure, and manage computing resources over the internet. Amazon Virtual Private Cloud (Amazon VPC) enables users to create logically isolated virtual networks in which cloud resources can be deployed and managed.

This project focuses on establishing private communication between two VPCs using VPC peering. Each VPC uses a separate IPv4 address range, allowing network resources to be organized independently. The peering connection enables communication between the networks when appropriate routing and security rules are configured.

The project uses three Amazon EC2 instances: a bastion host in a public subnet and two application instances in private subnets. The bastion host provides a controlled entry point for administrative access, while the application instances demonstrate communication through private IP addresses.

1.2 Problem Statement

Resources deployed in separate VPCs operate within separate virtual networks. To allow selected resources in these networks to communicate privately, the network administrator must establish a suitable connection and configure routing and access permissions correctly.

This project addresses that requirement by demonstrating how two VPCs can be connected through VPC peering. It also examines how incorrect or missing routes and security group rules can prevent communication, even when a peering connection exists.

1.3 Project Objectives

The main objectives of this project are:

1. To understand the purpose and structure of Amazon VPC.
2. To create or configure two VPCs with separate CIDR blocks.
3. To organize resources using public and private subnets.

4. To deploy a bastion host and two private EC2 application instances.
5. To establish a VPC peering connection between the two networks.
6. To configure route tables for communication in both directions.
7. To apply security group rules for controlled SSH and ICMP traffic.
8. To test network connectivity using private IP addresses.
9. To identify and troubleshoot connectivity problems.
10. To understand the importance of secure access and cloud resource cleanup.

1.4 Project Scope

The scope of the project includes VPC and subnet configuration, EC2 instance deployment, VPC peering, route table management, security group configuration, and connectivity testing.

The project also includes testing access to private instances through a bastion host and observing the effect of changing an ICMP security group rule. These activities help demonstrate the relationship between network routing and security controls.

1.5 Expected Outcomes

The expected outcome is a network design in which two VPCs can communicate through a peering connection using private IP addresses, provided the connection is active and the necessary routes and security group rules are in place.

The project is also expected to demonstrate controlled SSH access through the bastion host, successful connectivity tests when the network is configured correctly, and the ability to identify a security rule that blocks ICMP traffic.

Actual results should be confirmed using AWS Console evidence and terminal output.

1.6 Tools and Technologies Used

1. Amazon VPC: Creates isolated virtual networks.
2. Amazon EC2: Provides virtual servers for the bastion and application instances.
3. VPC Peering: Connects the two VPCs privately.
4. Route Tables: Direct traffic to the appropriate destination.
5. Security Groups: Control inbound traffic to EC2 instances.
6. SSH: Provides secure remote administrative access.
7. ICMP/Ping: Helps test network reachability.
8. AWS Management Console: Used to configure and inspect AWS resources.

Section Summary: This project provides practical learning in AWS cloud networking, private connectivity, access control, and network troubleshooting.

2. AWS Network Architecture

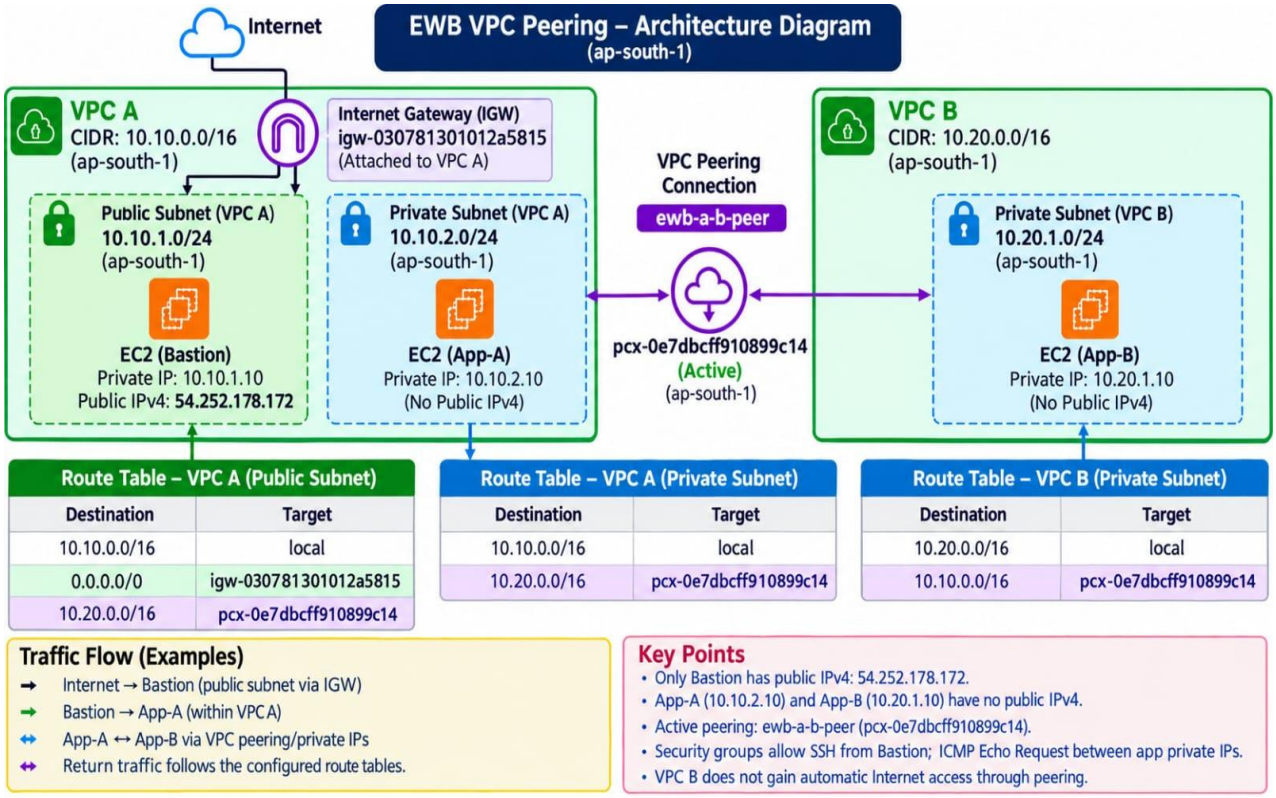
2.1 Architecture Overview

The AWS VPC Peering project consists of two Virtual Private Clouds (VPCs), three Amazon EC2 instances, public and private subnets, route tables, security groups, an internet gateway, and a VPC peering connection.

VPC A contains a public subnet and a private subnet. The public subnet hosts the bastion instance, which provides a controlled entry point for administrative access. App-A is deployed in the private subnet of VPC A.

VPC B contains a private subnet that hosts App-B. The application instance does not require a public IP address to communicate privately with resources in VPC A through the peering connection.

The two VPCs use separate IPv4 CIDR blocks to avoid address-range overlap and support private network communication.



3. Inventory Table:

Resource	Name	ID/Address
VPC A	ewb-vpc-a	vpc-0945ba1eea575f817 (10.10.0.0/16)
VPC B	ewb-vpc-b	vpc-05c4789f18a08d1a7 (10.20.0.0/16)
Public subnet	ewb-a-public	subnet-0bd52c5109e6a1604 (10.10.1.0/24)
Private subnet A	ewb-a-private	subnet-0ab7233afc2b91c17 (10.10.2.0/24)
Private subnet B	ewb-b-private	subnet-0cc3a9986106e6b46 (10.20.1.0/24)
Internet gateway	ewb-igw-a	igw-030781301012a5815
Peering connection	ewb-a-b-peer	pcx-0e7dbcff910899c14

Bastion	ewb-bastion	i-04e93dde3e5d10bc7 /10.10.1.10/ public54.252.178.172
App-A	ewb-app-a	i-0f38b2abcc6d36005/ 10.10.2.10/no public IP
App-B	ewb-app-b	i-0144439db96c10da0/ 10.20.1.10/no public IP
Security groups	ewb-sg-bastion, ewbsg-app-a, ewb-sgapp-b	sg-0e8e8554875cf1d99 is bastion, sg-0c0d9ef06c49670bd is app-a, sg-024c74a7434595e08 is app-b
Route tables	ewb-rt-a-public,ewbrt-a-private,ewb-rt-bprivate	rtb-0b2ee3bb79fd79e53 is public-a, rtb-0853ed9a5aa2259a6 is private a, rtb-03ec9103f22489a36 is private-b
Key pair	ewb-lab	

3.1 Resource Inventory Summary

The resource inventory identifies the main AWS resources used in the VPC Peering project, including VPCs, subnets, EC2 instances, and the VPC peering connection. Each resource has a specific role in the network architecture.

The two VPCs use separate CIDR blocks: "10.10.0.0/16" for VPC A and "10.20.0.0/16" for VPC B. The bastion host is located in the public subnet of VPC A, while App-A and App-B are deployed in private subnets.

The inventory provides a reference for checking resource configuration, private IP addresses, and AWS resource IDs during connectivity testing and troubleshooting.

Note: Resource names, IP addresses, and resource IDs in this report should match the actual configuration in your AWS account.

5. Network and Security Configuration

5.1 Network Configuration

Network configuration determines how resources communicate within and between the two VPCs. In this project, VPC A contains a public subnet and a private subnet, while VPC B contains a private subnet.

The internet gateway attached to VPC A supports internet connectivity for resources that have the appropriate routes and security rules. VPC B is designed to keep App-B private and communicate with VPC A through the VPC peering connection.

5.2 Route Table Configuration

Route tables define the paths used by network traffic.

1. Public Route Table ("ewb-rt-a-public"): Contains a local route for VPC A, a default route to the internet gateway, and a route to VPC B through the peering connection when configured.
2. Private Route Table A ("ewb-rt-a-private"): Contains a local route for VPC A and a route to "10.20.0.0/16" through the peering connection.
3. Private Route Table B ("ewb-rt-b-private"): Contains a local route for VPC B and a route to "10.10.0.0/16" through the peering connection.

The routes to the other VPC must be configured correctly for communication to work in both directions.

5.3 Security Group Configuration

Security groups act as virtual firewalls for EC2 instances. They control which inbound connections are allowed.

The project uses the following intended rules:

1. "ewb-sg-bastion": Allows SSH traffic on TCP port 22 from the administrator's public IPv4 address using a "/32" CIDR.
2. "ewb-sg-app-a": Allows SSH from the bastion host and ICMP Echo Requests from App-B ("10.20.1.10/32").

3. "ewb-sg-app-b": Allows SSH from the bastion host and ICMP Echo Requests from App-A ("10.10.2.10/32").

Only the required traffic should be allowed. The actual inbound rules should be checked in the AWS Console before recording them as completed.

5.4 Importance of Network Security

Correct routing makes communication possible, while security groups control whether the traffic is permitted. Both configurations must be correct for successful connectivity.

Using a bastion host provides a controlled entry point for administration, and keeping application instances in private subnets reduces their direct exposure to the public internet.

5.5 Configuration Evidence

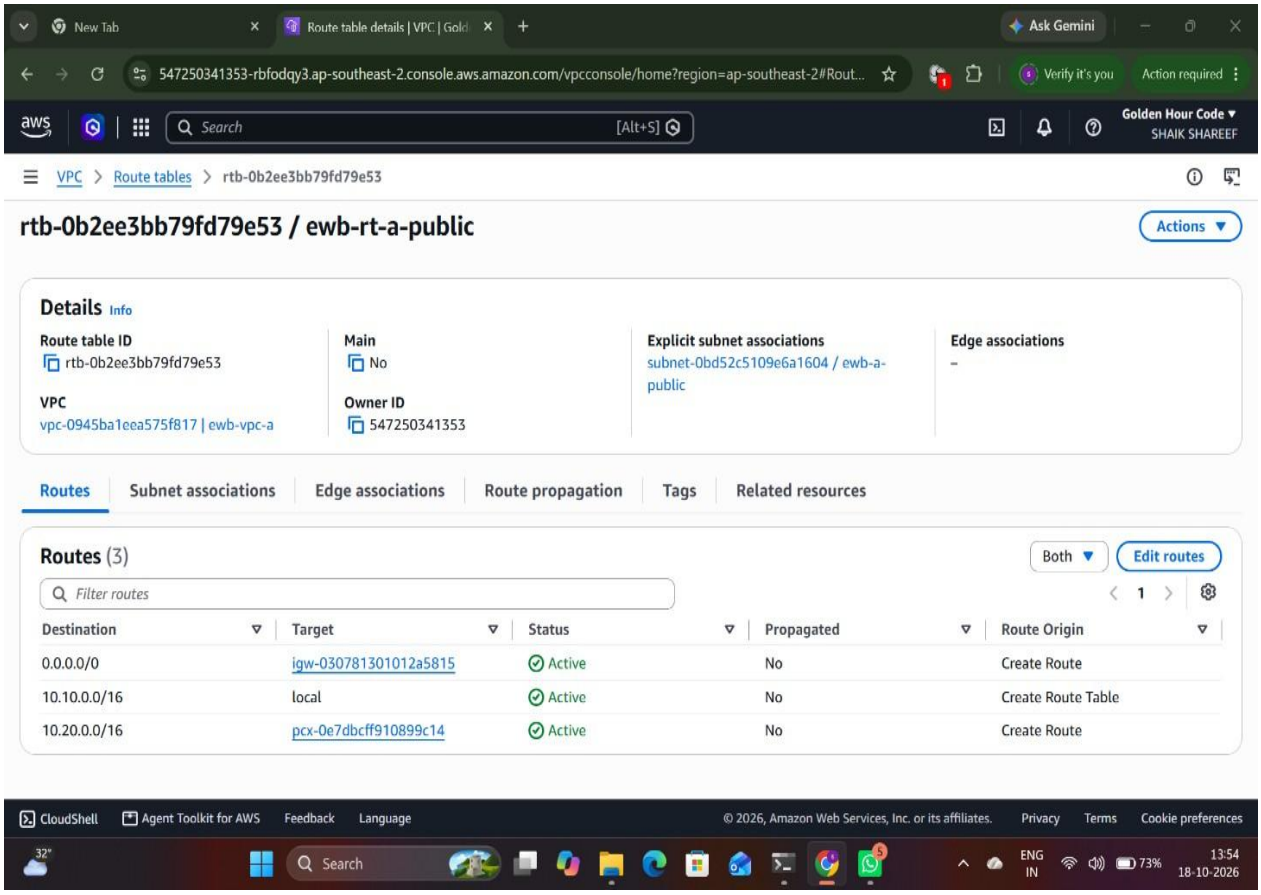


Fig 1: Public route tables for VPC A

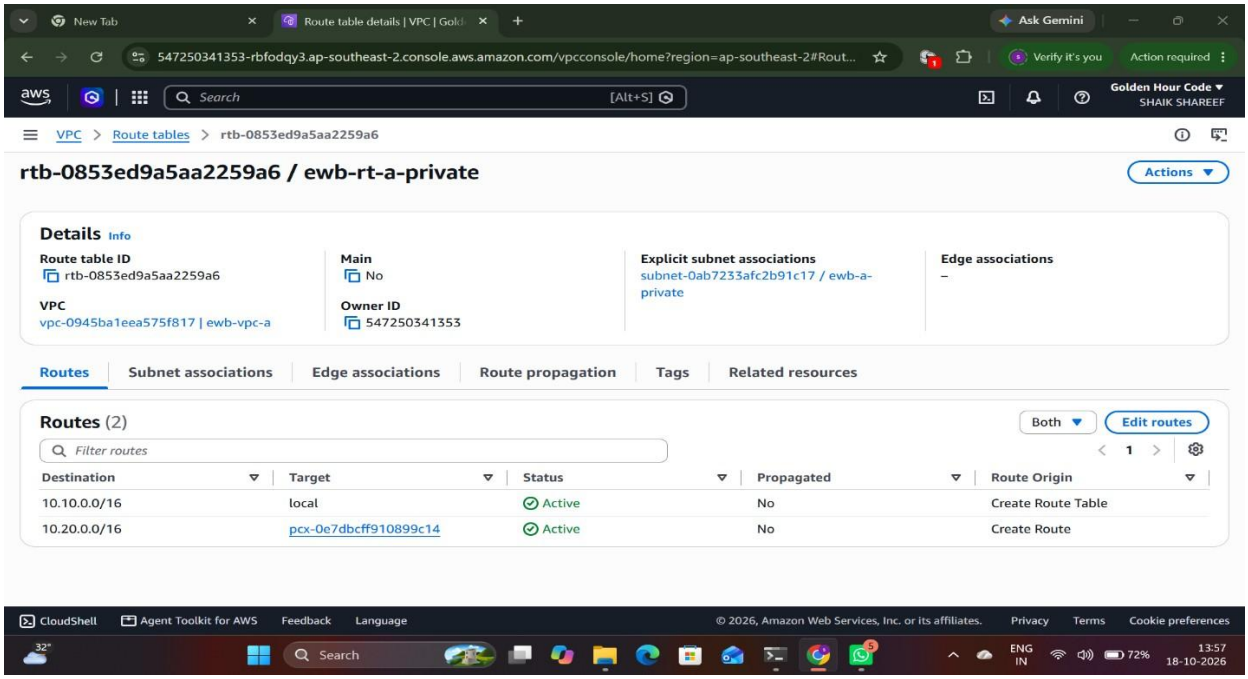


Fig 2: private route table for VPC A

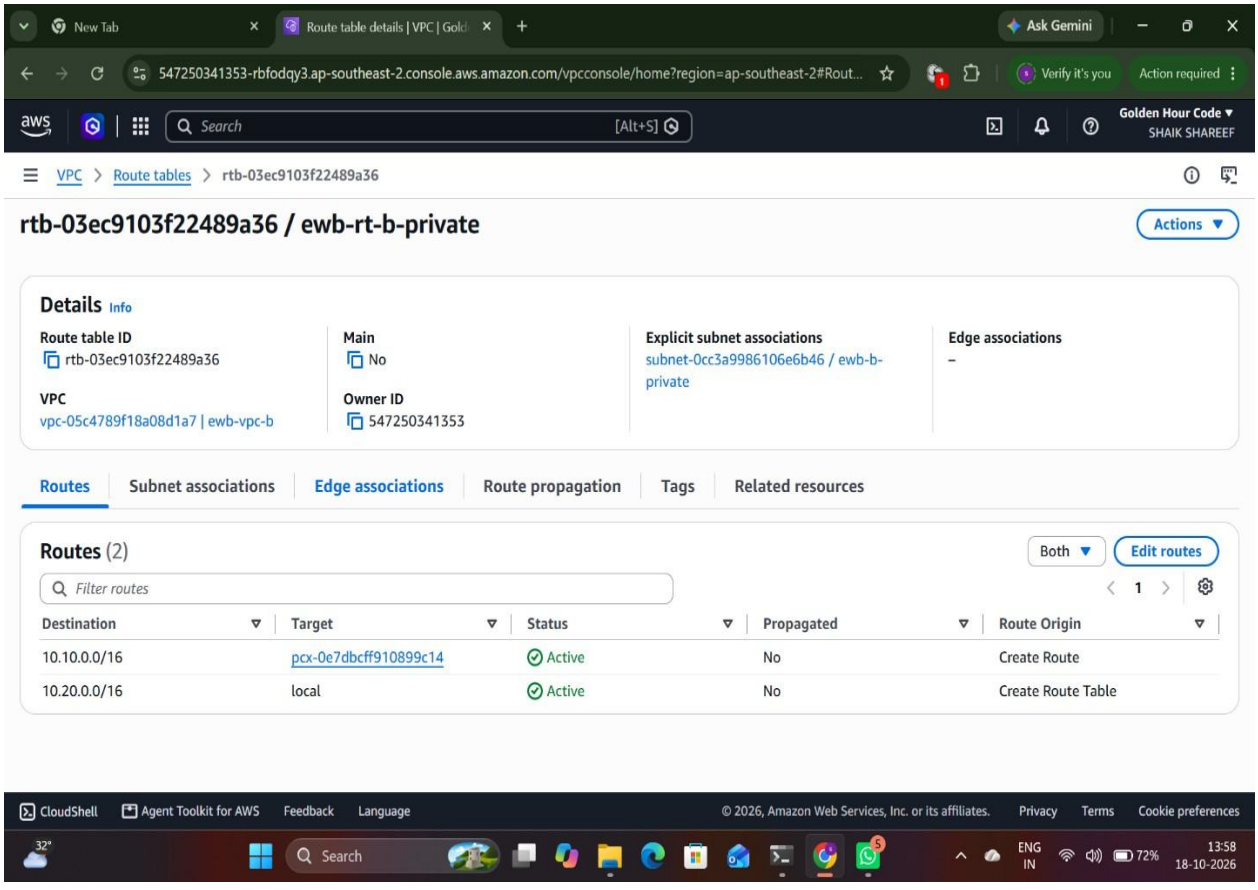


Fig 3: Private route table for VPC B

6. EC2 Instances and Security Groups

6.1 Introduction

Amazon Elastic Compute Cloud (Amazon EC2) provides virtual servers that can run applications and perform computing tasks in the AWS cloud. In this project, three EC2 instances are used to demonstrate secure access and private communication between two Virtual Private Clouds (VPCs).

The instances include one bastion host in the public subnet of VPC A and two application instances in private subnets. Each instance has a specific role in the network architecture.

6.2 EC2 Instance Details

1. Bastion Host — "ewb-bastion"

The bastion host is located in the public subnet of VPC A. Its private IP address is "10.10.1.10". It provides a controlled entry point for an administrator to access private instances through SSH. A public IPv4 address is required for direct SSH access from the internet.

2. Application Instance A — "ewb-app-a"

App-A is located in the private subnet of VPC A and uses the private IP address "10.10.2.10". It represents an application server within the first VPC and communicates with App-B through the VPC peering connection when routing and security rules permit the traffic.

3. Application Instance B — "ewb-app-b"

App-B is located in the private subnet of VPC B and uses the private IP address "10.20.1.10". It represents an application server in the second VPC and is designed to communicate with App-A using private IP addressing.

6.3 Security Group Rules

Security groups act as virtual firewalls that control inbound traffic to EC2 instances.

1. Bastion security group: Allows SSH on TCP port 22 from the administrator's public IPv4 address, restricted to a "/32" CIDR.

2. App-A security group: Allows SSH from the bastion host and ICMP Echo Requests from App-B.
3. App-B security group: Allows SSH from the bastion host and ICMP Echo Requests from App-A.

These rules are intended to permit the traffic required for administration and connectivity testing while limiting unnecessary access.

6.4 Security Best Practices

The application instances should remain in private subnets without public IP addresses. Administrative access should be restricted to trusted sources, and security group rules should allow only the ports and traffic types required for the project.

Private keys, passwords, and access credentials must be stored securely and must not be included in the project report.

6.5 Evidence Screenshots

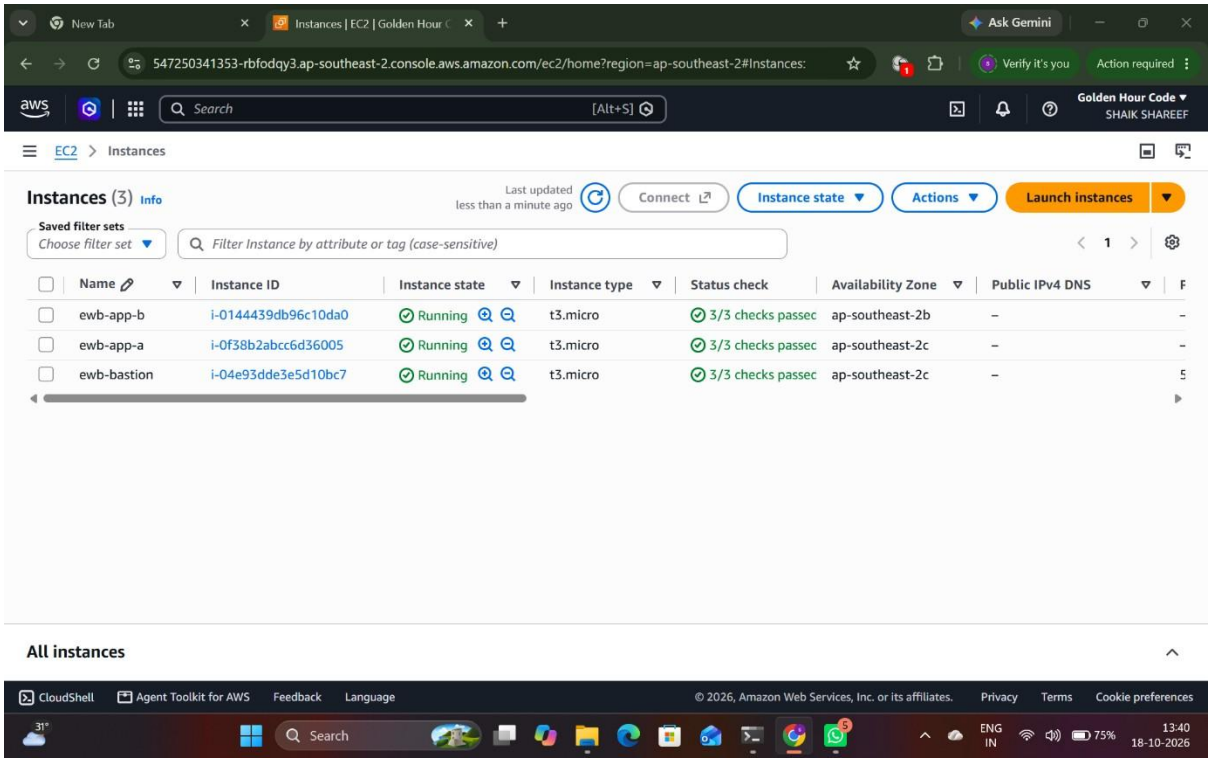


Fig 4: EC2 Three Instance Name and Status

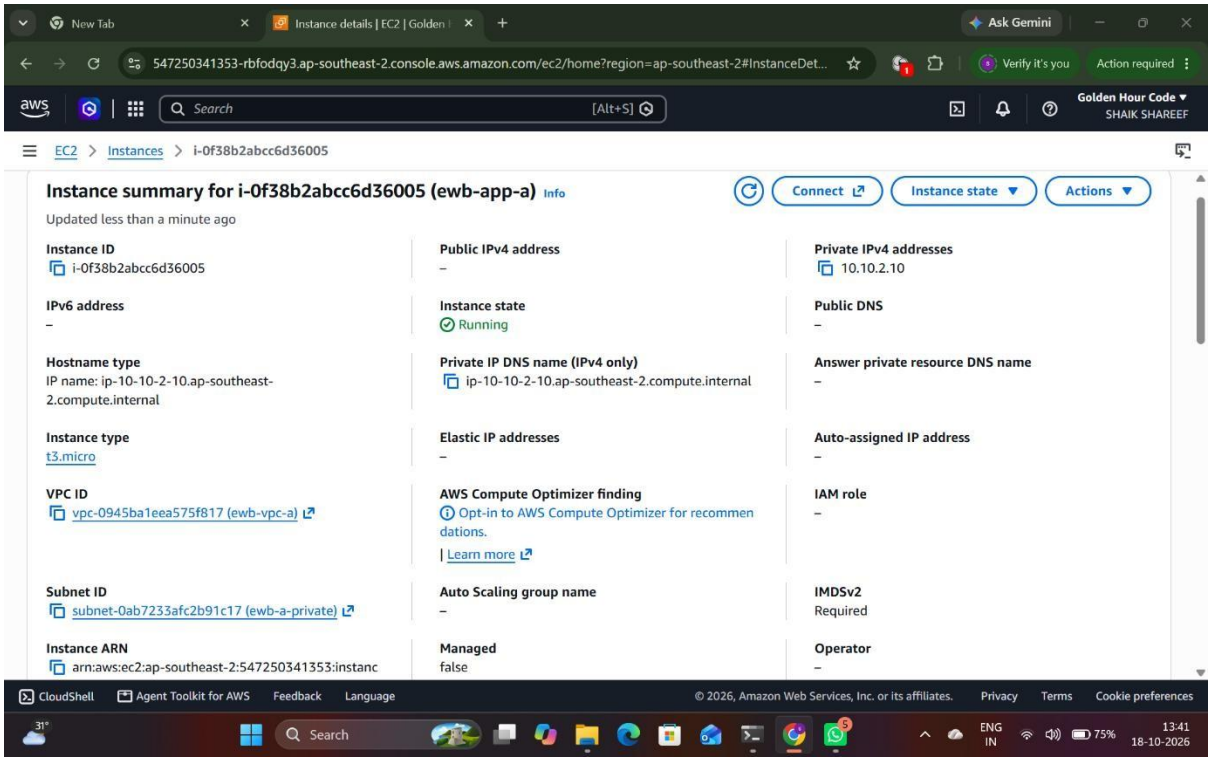


Fig 5: Details of ewb-app-a instance

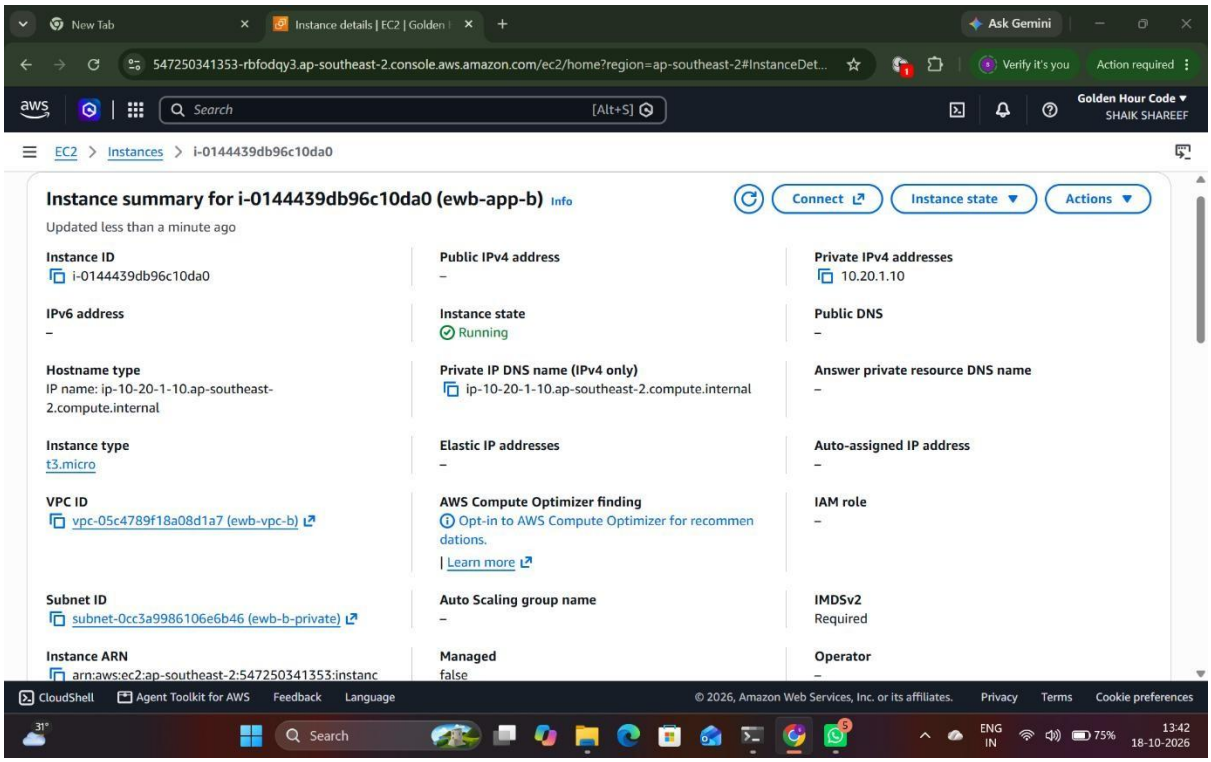


Fig 6: Details of ewb-app-b instance

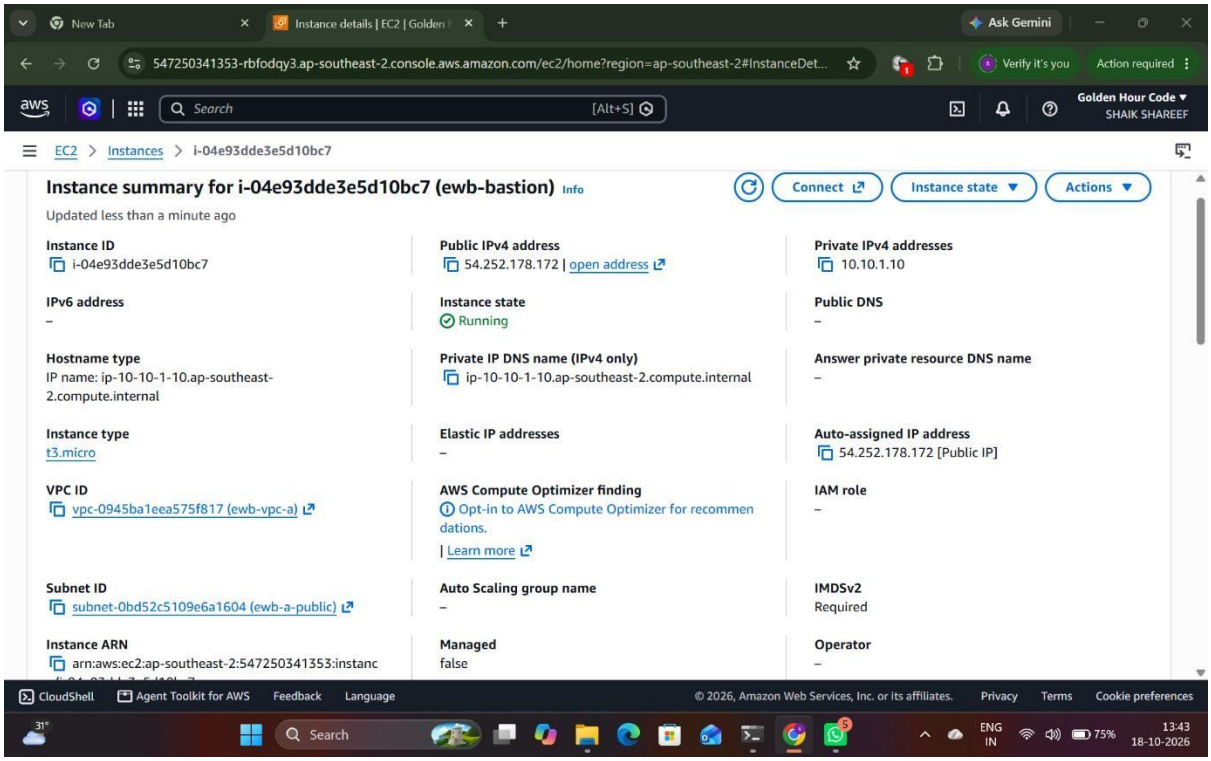


Fig 7: Details of ewb-bastion instance

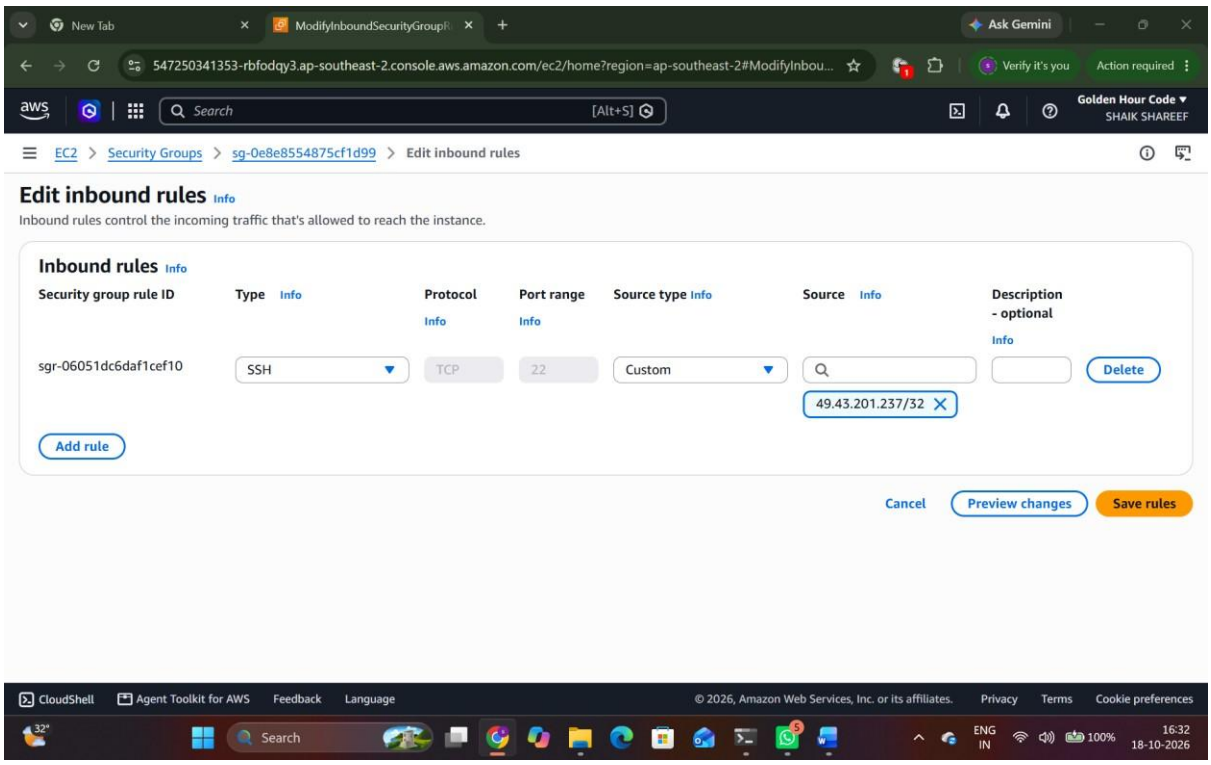


Fig 8: Security Groups of ewb-sg-bastion Inbound rule

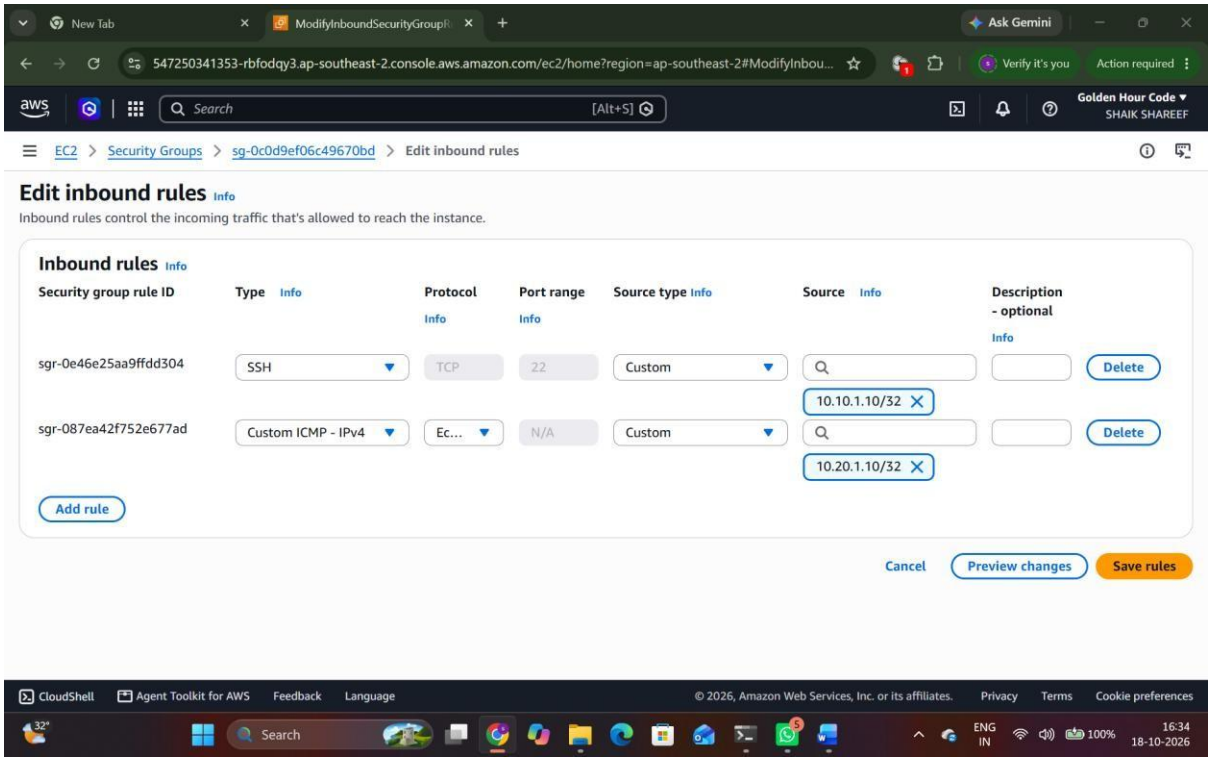


Fig 9: Security Groups of ewb-sg-app-a Inbound rule

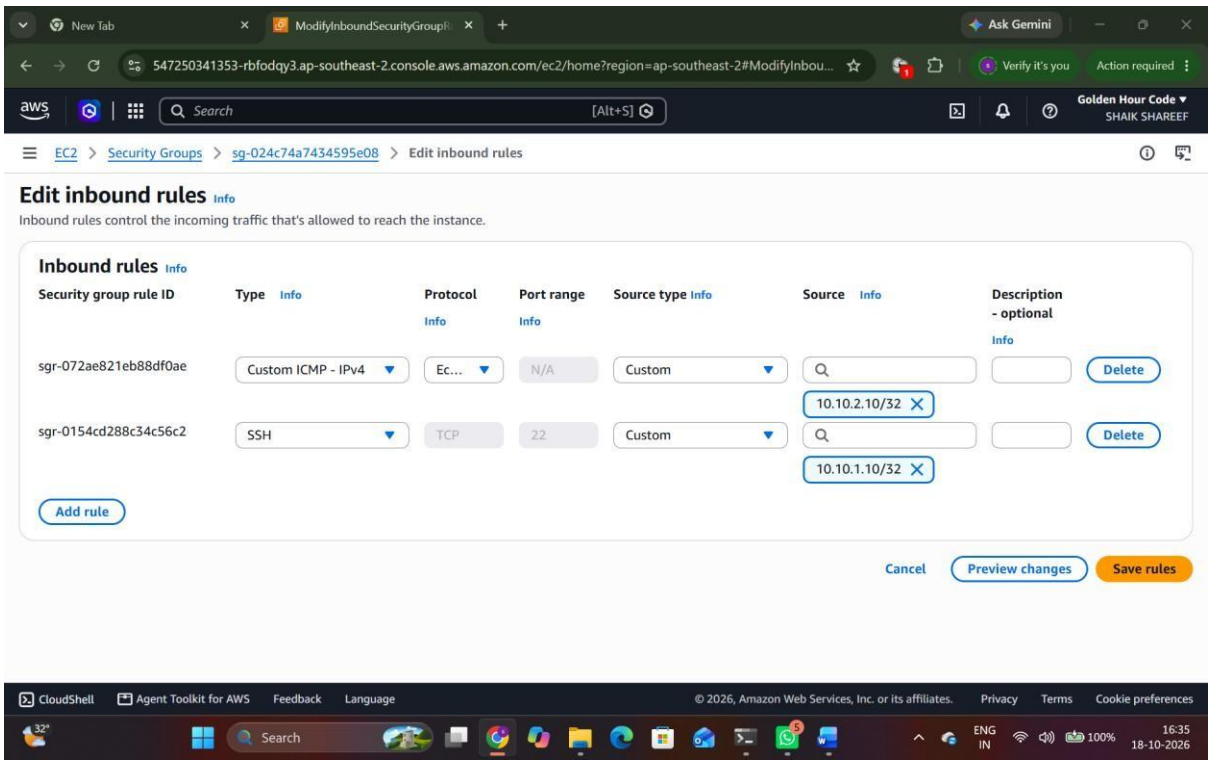


Fig 10: Security Groups of ewb-sg-app-b Inbound rule

7. VPC Peering Connection and Route Table Configuration

7.1 Introduction

Amazon VPC Peering allows two Virtual Private Clouds (VPCs) to communicate privately using their private IP addresses. In this project, VPC A and VPC B are connected through a peering connection named "ewb-a-b-peer".

VPC A uses the CIDR block "10.10.0.0/16", while VPC B uses "10.20.0.0/16". These separate CIDR blocks allow the networks to communicate without overlapping address ranges.

7.2 VPC Peering Connection

The peering connection provides a network path between the two VPCs. The connection must be created and accepted before it can be used.

Peering connection name: "ewb-a-b-peer"

Requester VPC: "ewb-vpc-a"

Accepter VPC: "ewb-vpc-b"

Connection status: Record the actual status shown in the AWS Console. The connection must show Active before it can be used for communication.

7.3 Route Table Configuration

Route tables determine how network traffic reaches its destination. Routes to the remote VPC must point to the VPC peering connection.

The required routes are shown below.

Route table| Destination CIDR| Target

1. "ewb-rt-a-private"| "10.10.0.0/16"| Local
2. "ewb-rt-a-private"| "10.20.0.0/16"| VPC peering 3. "ewb-rt-b-private"| "10.20.0.0/16"| Local
4. "ewb-rt-b-private"| "10.10.0.0/16"| VPC peering
5. "ewb-rt-a-public"| "0.0.0.0/0"| Internet gateway
6. "ewb-rt-a-public"| "10.20.0.0/16"| VPC peering

The local routes are automatically present for each VPC's own CIDR block. The peering routes allow traffic to be directed to the other VPC. The public route table also needs a route to VPC B if the bastion host is to reach App-B through the peering connection.

7.4 Importance of Bidirectional Routing

For App-A and App-B to communicate successfully, both VPCs need suitable routes. The route table in VPC A must direct traffic for VPC B through the peering connection, and the route table in VPC B must provide a return path to VPC A.

Even when the peering connection is active, missing or incorrect routes can prevent communication. Security groups must also permit the required traffic.

7.5 Verification in AWS Console

The configuration can be verified using the following steps:

1. Open the AWS Management Console and search for VPC.
2. Select Peering connections from the left navigation menu.
3. Find "ewb-a-b-peer" and check its status.
4. Open Route tables.
5. Select each relevant route table and inspect its routes.
6. Confirm that the destination CIDRs and route targets match the intended configuration.

7.6 Evidence Screenshots

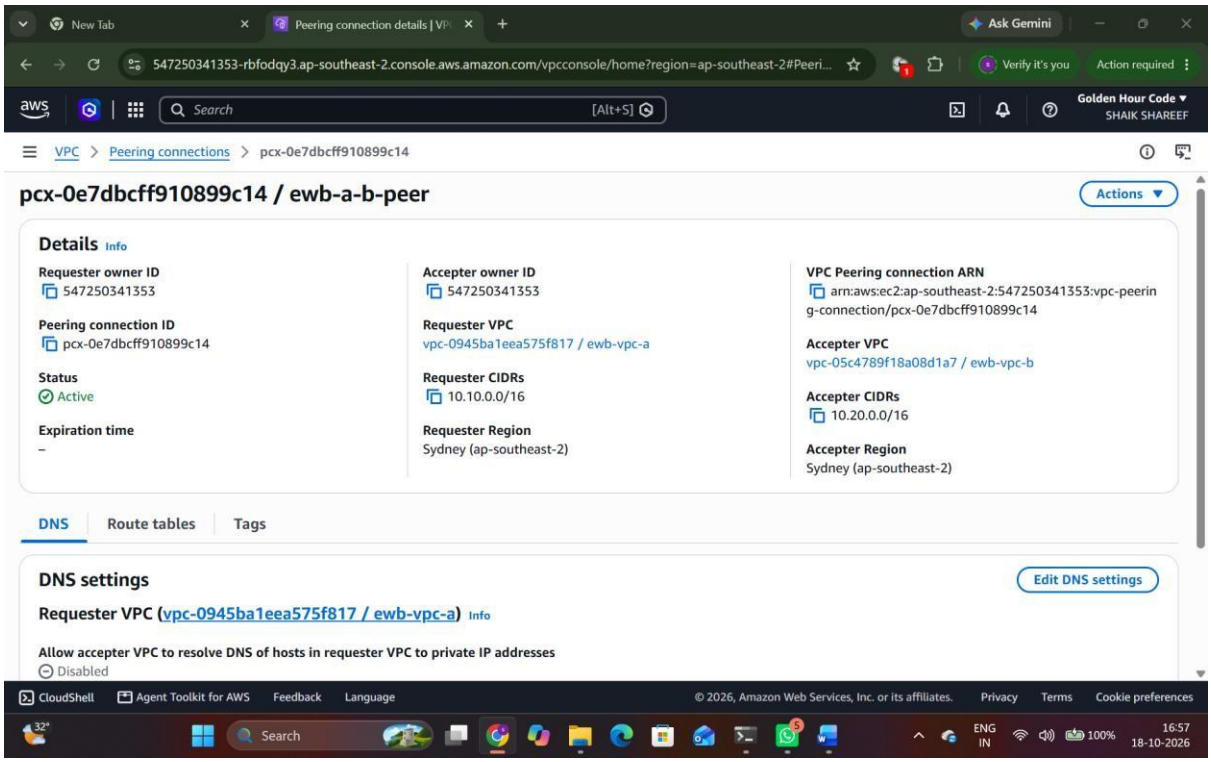


Fig 11: VPC peering connection Status

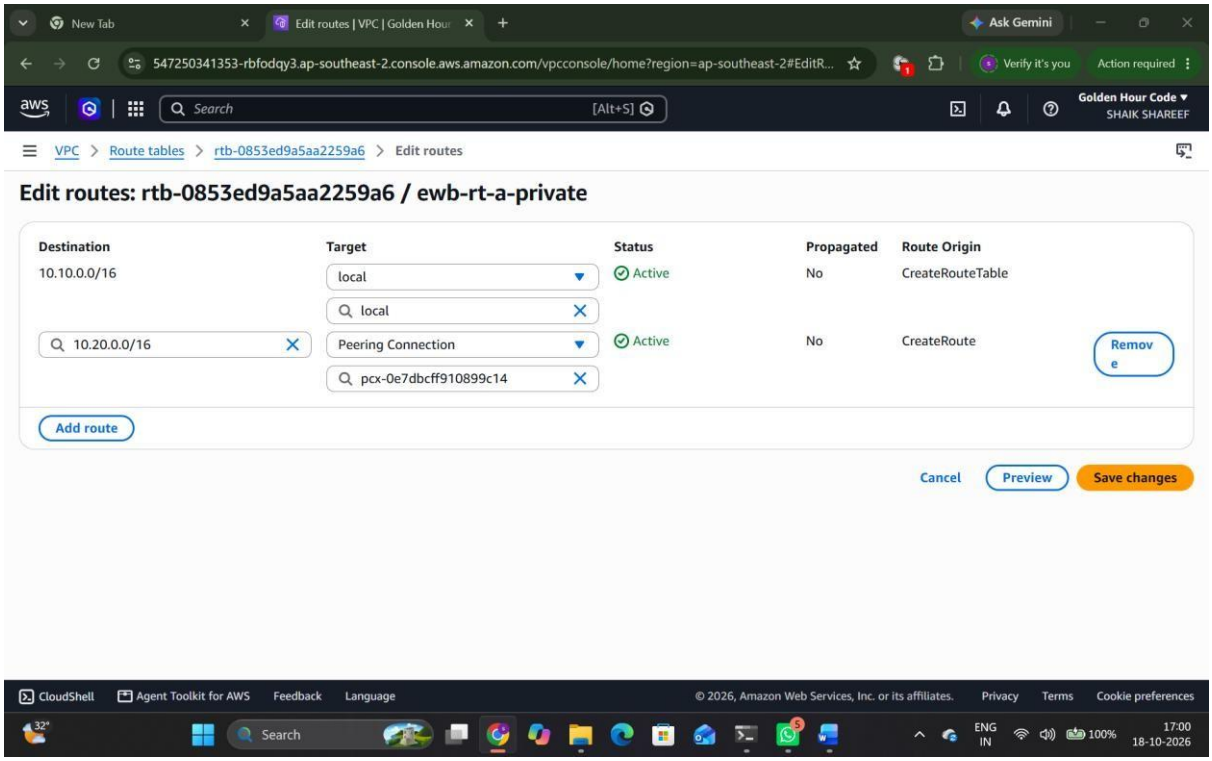


Fig 12: Route table VPC A's private subnets

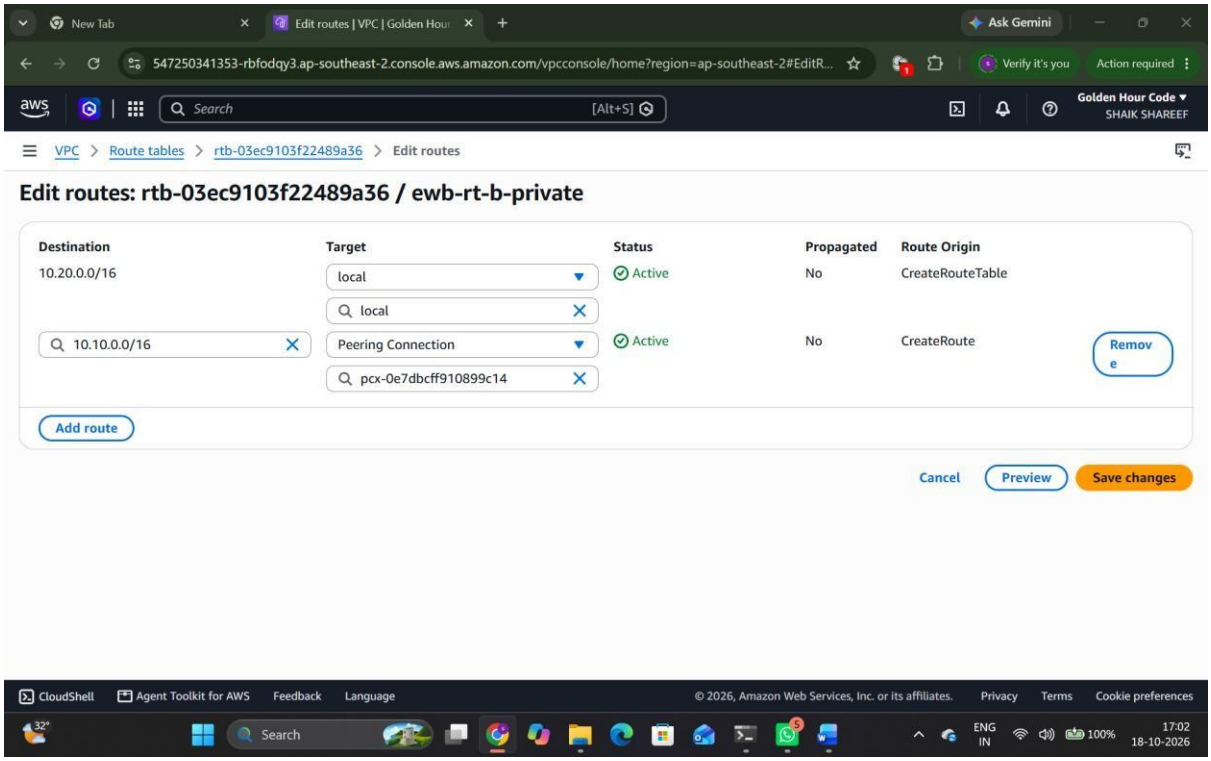


Fig 13: Route table for VPCB's private subnets

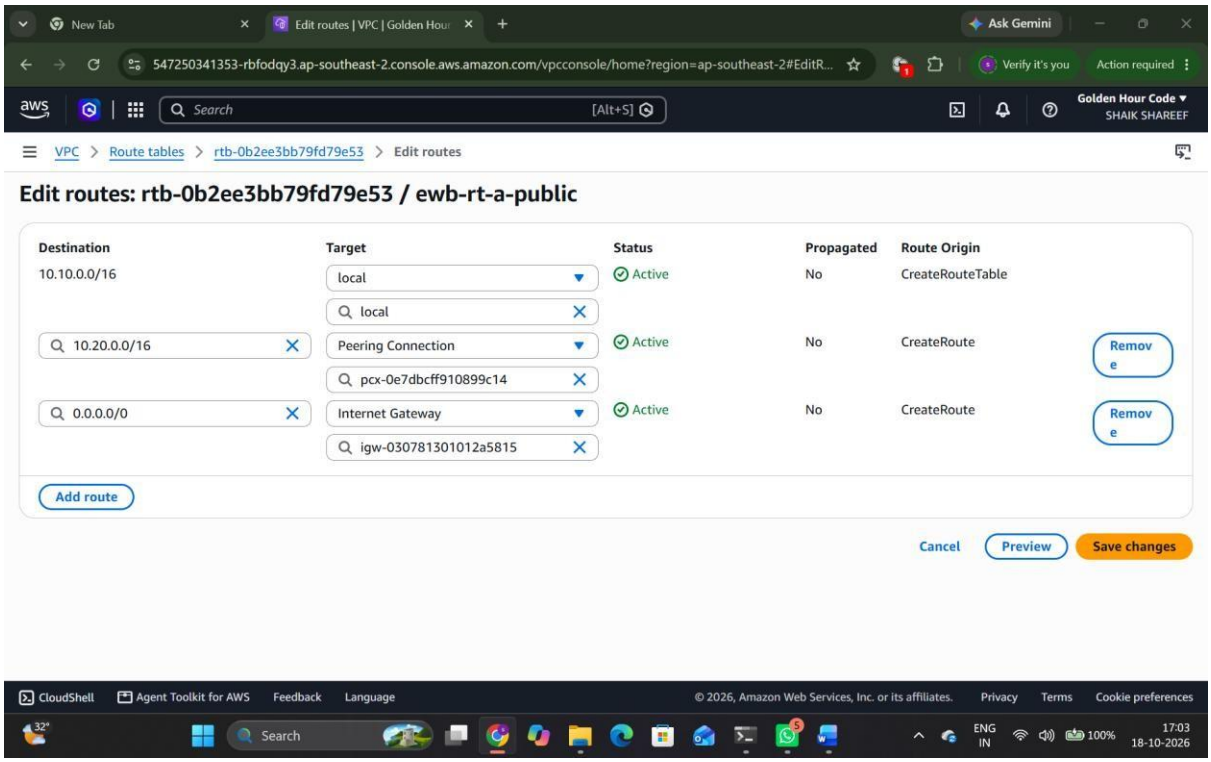


Fig 14: Route table for VPC A's public subnets

8. Connectivity Testing and SSH Access

8.1 Introduction

Connectivity testing is an important step in verifying an AWS network configuration. In this project, ping and SSH are used to examine communication between EC2 instances and access to private resources.

The tests focus on communication between App-A in VPC A and App-B in VPC B. The results depend on the VPC peering connection, route tables, security group rules, and the configuration of the EC2 instances.

8.2 Baseline Connectivity Test

The baseline test checks connectivity before the required peering routes are configured.

From App-A, run the following command:

ping 10.20.1.10

If the peering routes are not configured, communication between the two VPCs is expected to fail. Record the actual output and capture a screenshot of the test.

8.3 App-A to App-B Connectivity Test

After the peering connection is active and the required routes and security group rules are configured, test communication from App-A to App-B.

Run the following command from App-A:

ping 10.20.1.10

Successful replies indicate that App-A can reach App-B using its private IP address and that the required network and security settings permit the tested traffic.

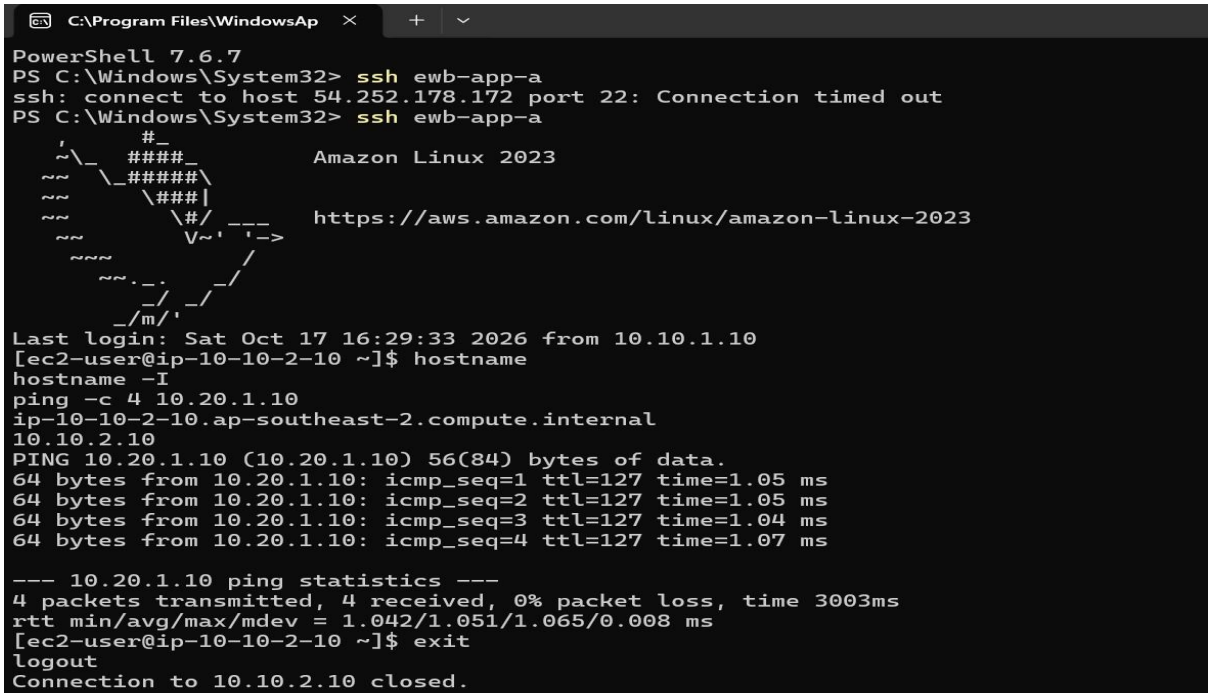


Fig 15: Successful Connectivity from App-A to App-B

8.4 App-B to App-A Connectivity Test

To verify communication in the reverse direction, run the following command from App-B:

ping 10.10.2.10

If replies are received, the test demonstrates that App-B can reach AppA using its private IP address.

If the test fails, check the route tables, security group rules, instance status, and network configuration.

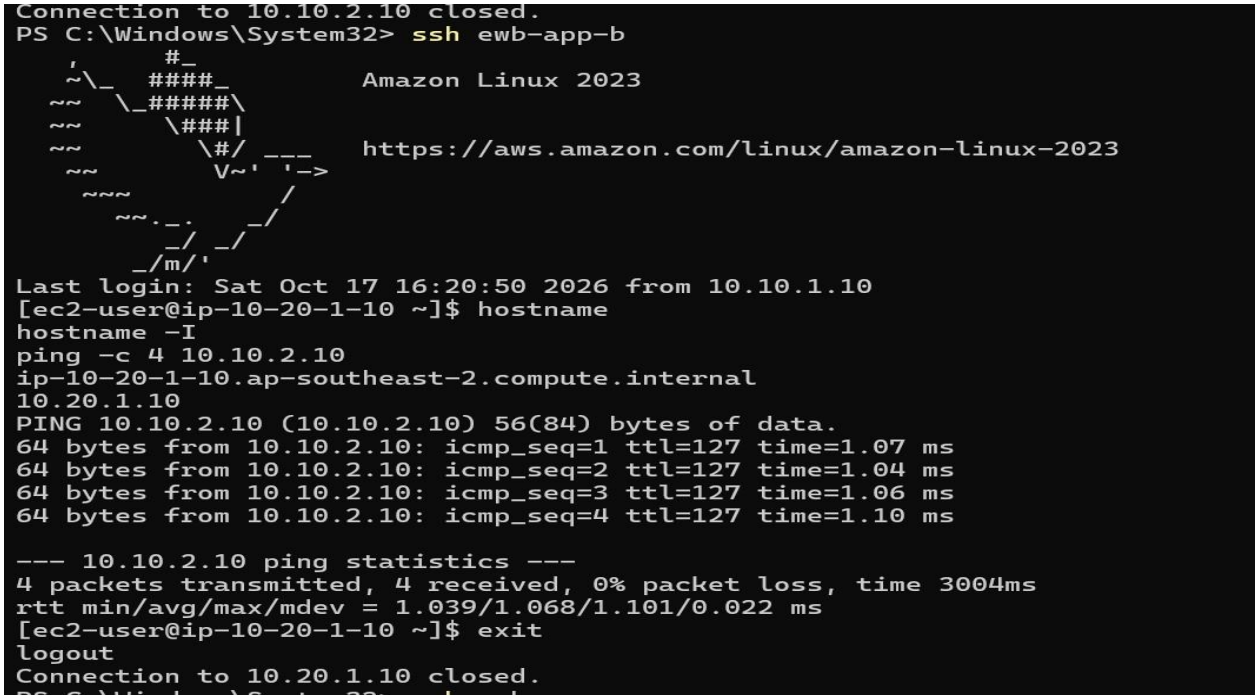


Fig 16: Successful Connectivity from App-B to App-A

8.5 SSH Access Through the Bastion Host

A bastion host provides a controlled entry point for accessing private EC2 instances. SSH ProxyJump can be used to connect to App-B through the bastion host.

Example command from a laptop with the appropriate SSH client and key configured:

ssh -J ec2-user@<54.252.178.172ec2-user@10.20.1.10

Replace "54.252.178.172" with the actual public IP address of the bastion host. Use the correct SSH username and key configuration for your EC2 operating system.

The bastion host must be reachable from the laptop, and its security group must permit authorized SSH traffic. App-B must permit SSH from the bastion host.

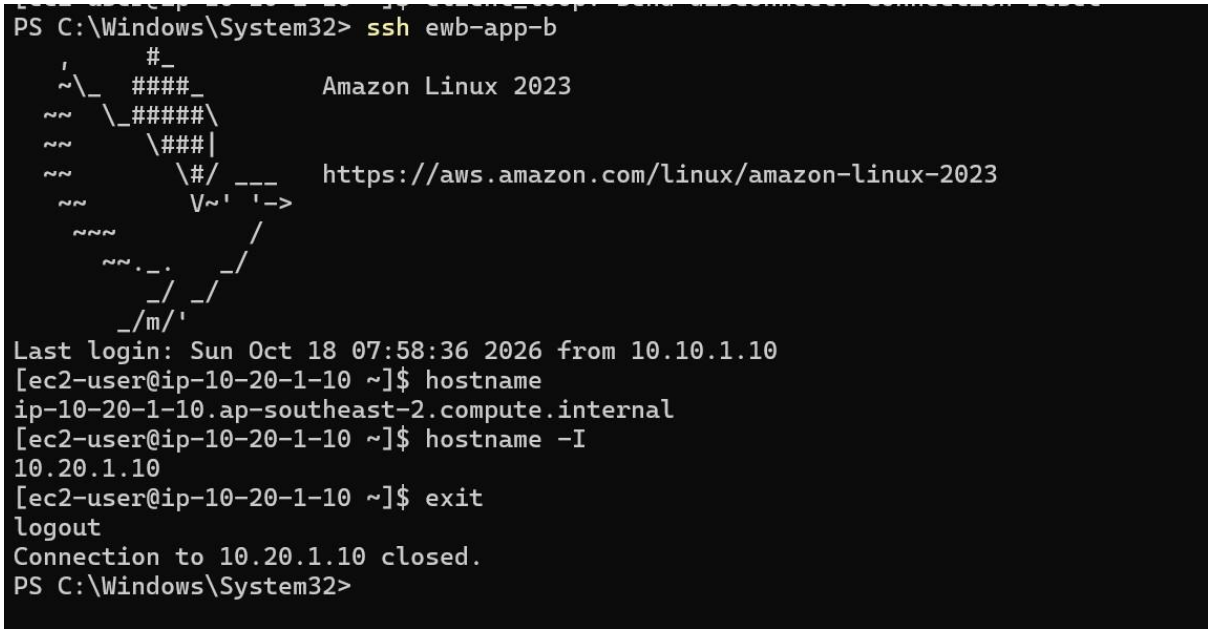


Fig 17: Successful SSH access to App-B through the bastion host

8.6 Ping failure and recovery and troubleshooting

8.7 Ping Failure

During testing, App-A ("10.10.2.10") attempted to communicate with AppB ("10.20.1.10"). The test showed 100% packet loss, indicating that no ping replies were received.

8.8 Connectivity Recovery

After further testing, the ping test succeeded with four packets received and 0% packet loss. This confirmed successful communication between App-A and App-B using their private IP addresses.

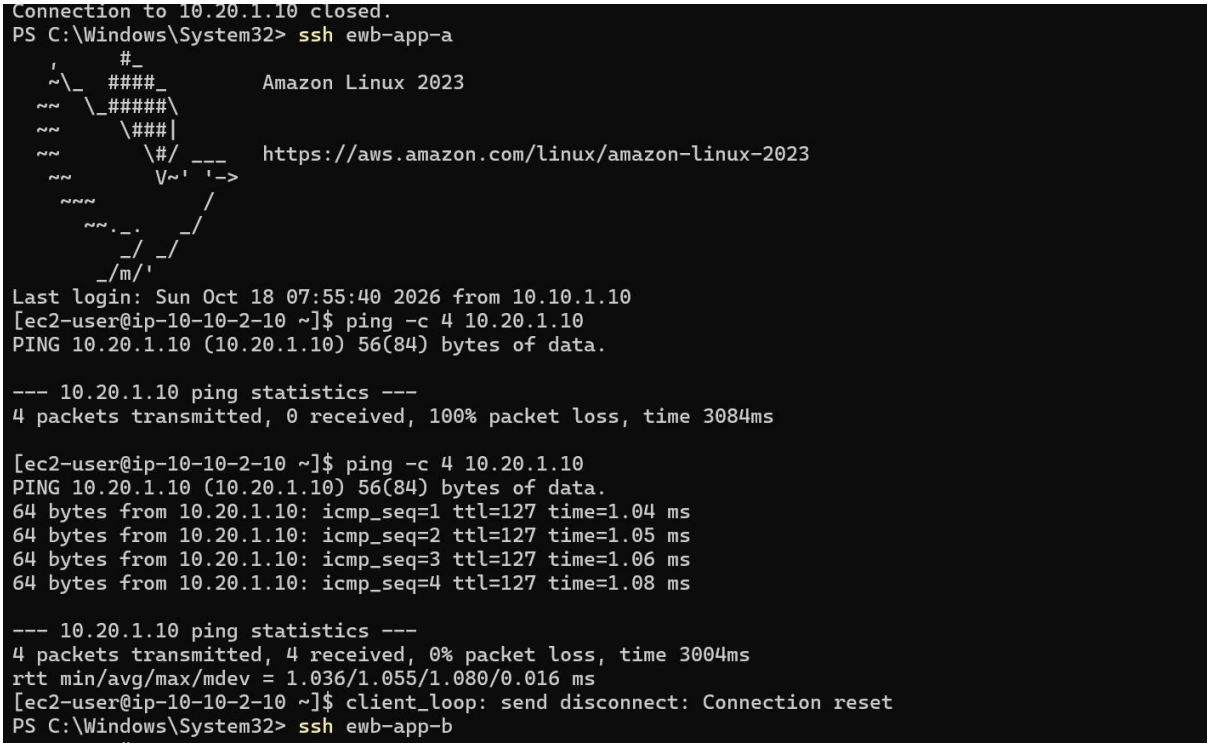


Fig 18: Connectivity failure and subsequent recovery

10. Results and Lessons Learned

10.1 Project Results

The AWS VPC Peering project was successfully tested for private communication between two VPCs.

1. Created two VPCs with separate IP address ranges.
2. Configured public and private subnets.
3. Established a VPC peering connection.
4. Configured route tables and security groups.
5. Verified SSH access to private EC2 instances through the bastion host.
6. Successfully tested ping connectivity between App-A and App-B with 0% packet loss.

10.2 Lessons Learned

Through this project, I learned how to configure VPC peering, manage route tables, apply security group rules, and connect to private EC2 instances using a bastion host. I also learned how to test network connectivity and troubleshoot connection failures.

11. Resource Cleanup and Final Verification

11.1 Resource Cleanup

After completing the project, AWS resources should be reviewed to avoid unnecessary charges. The EC2 instances, VPC peering connection, subnets, route tables, internet gateway, and security groups should be removed when they are no longer required.

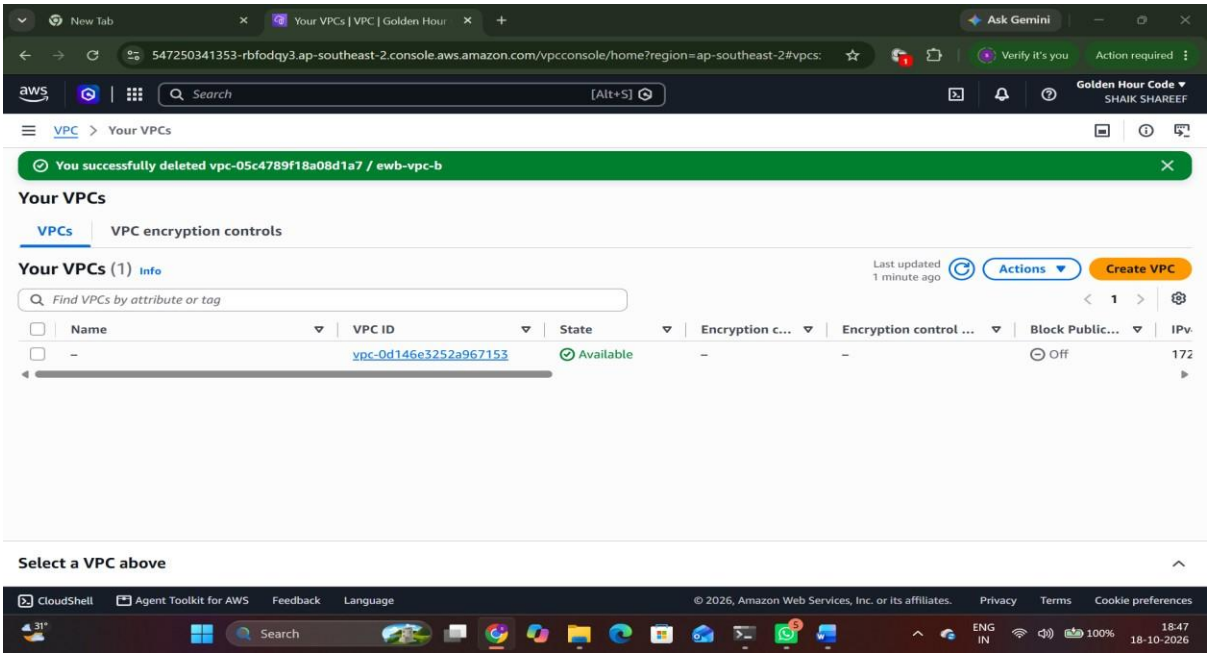


Fig 20: Deleted the VPC A's and VPC B's

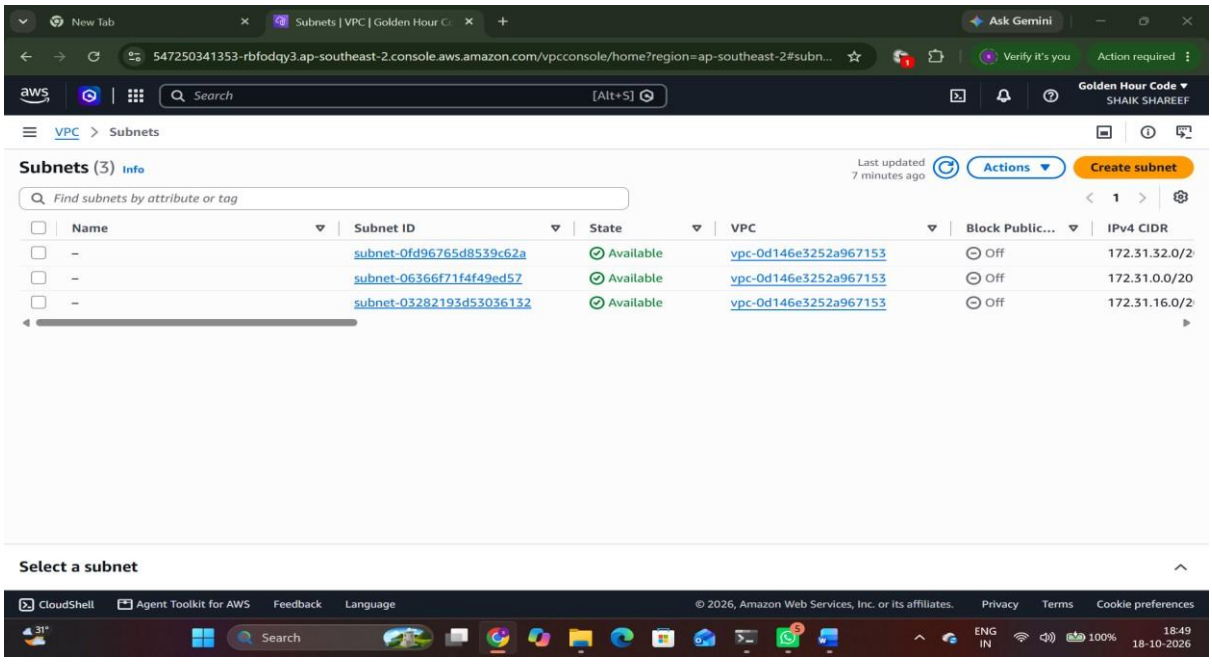


Fig 21: Deleted the All subnets

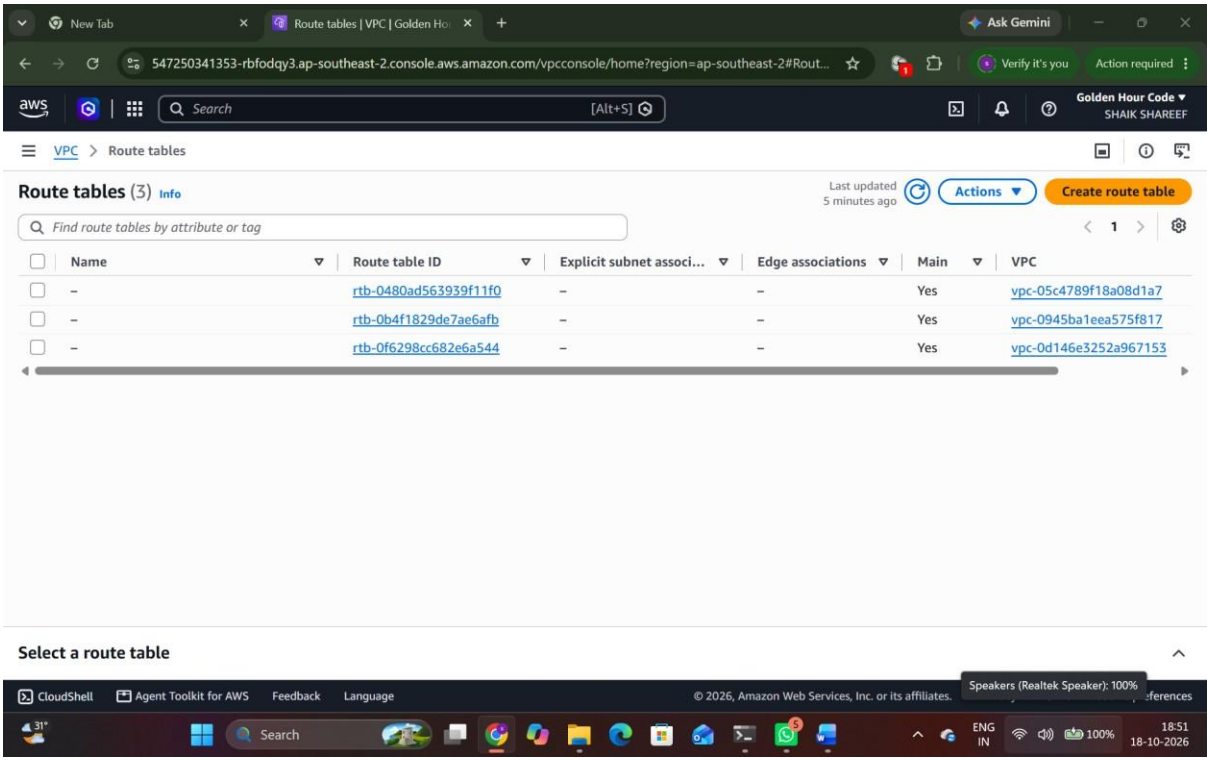


Fig 22: Deleted the All-route Tables

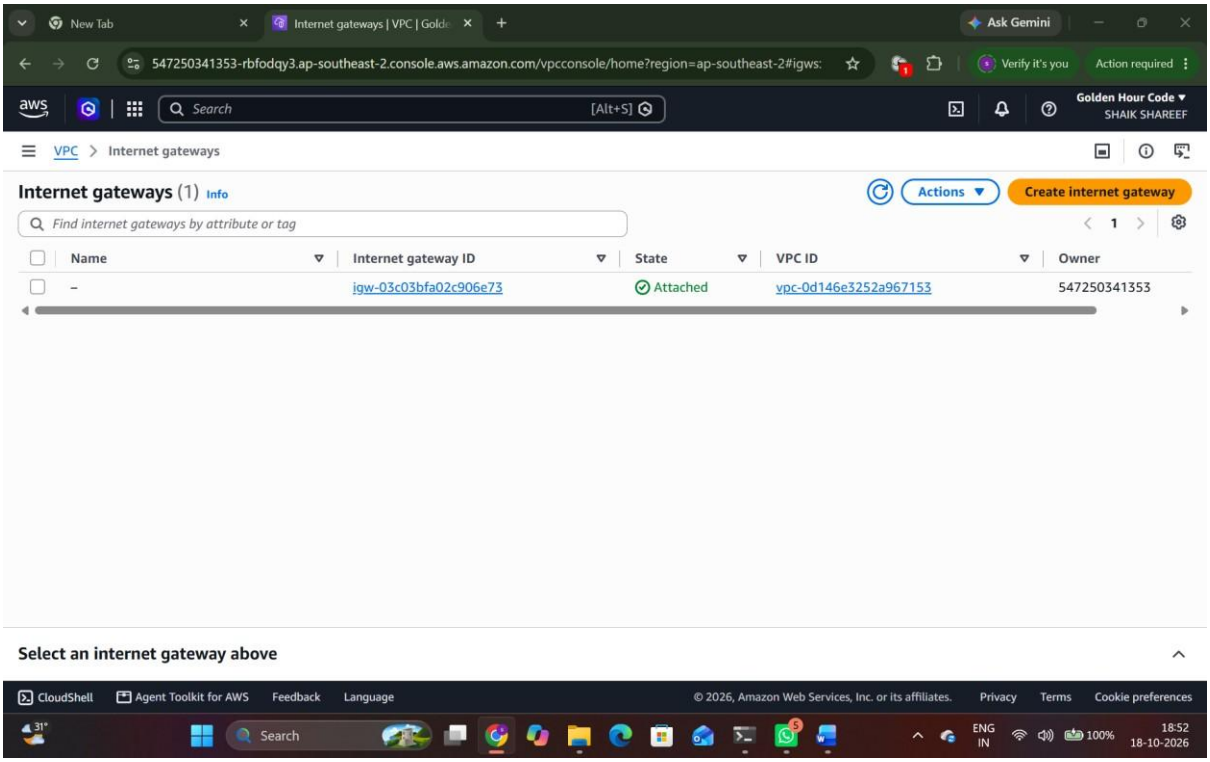


Fig 23: Deleted the Internet gateway

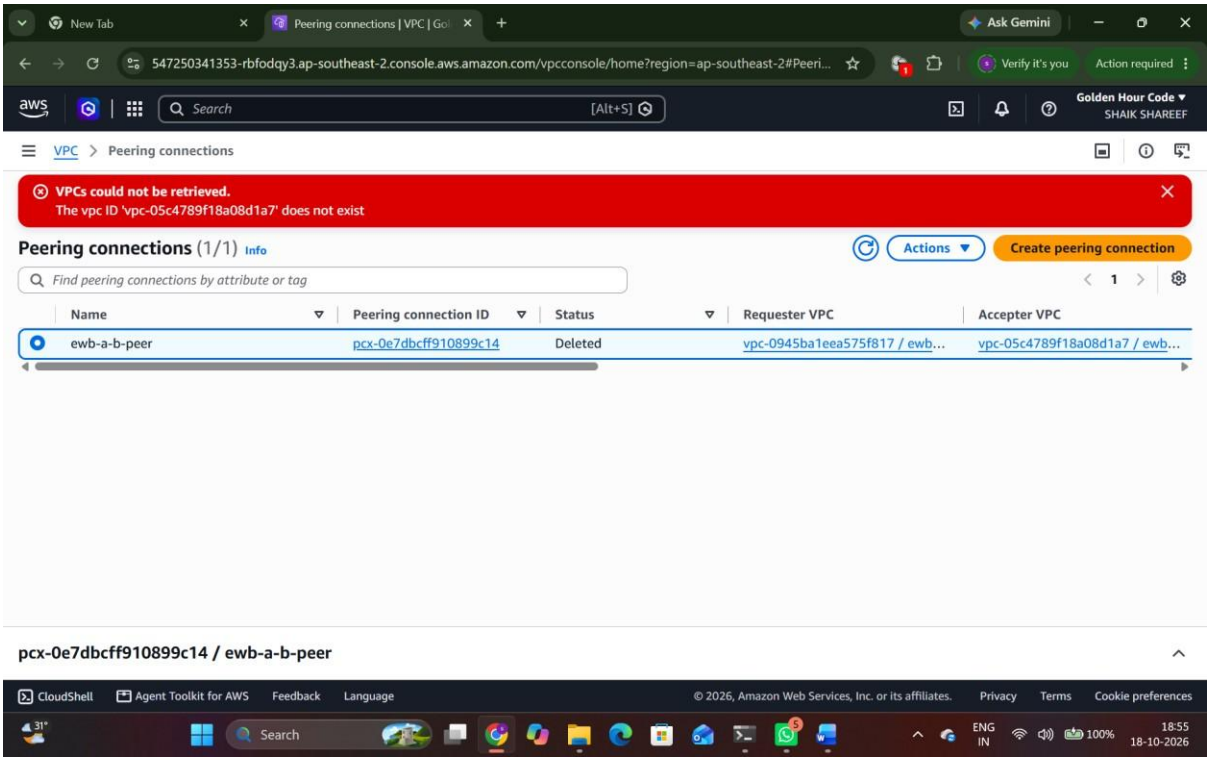


Fig 26: Deleted the Perring connection

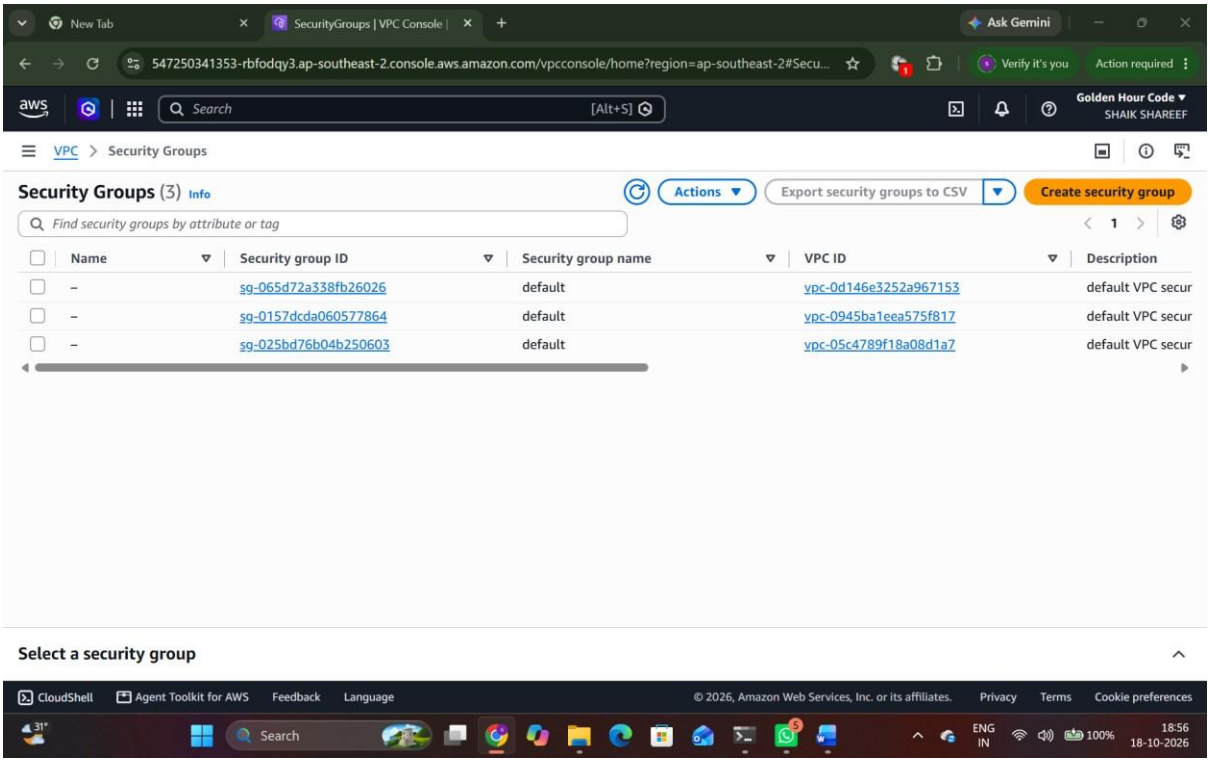


Fig 27: Deleted the all-security groups

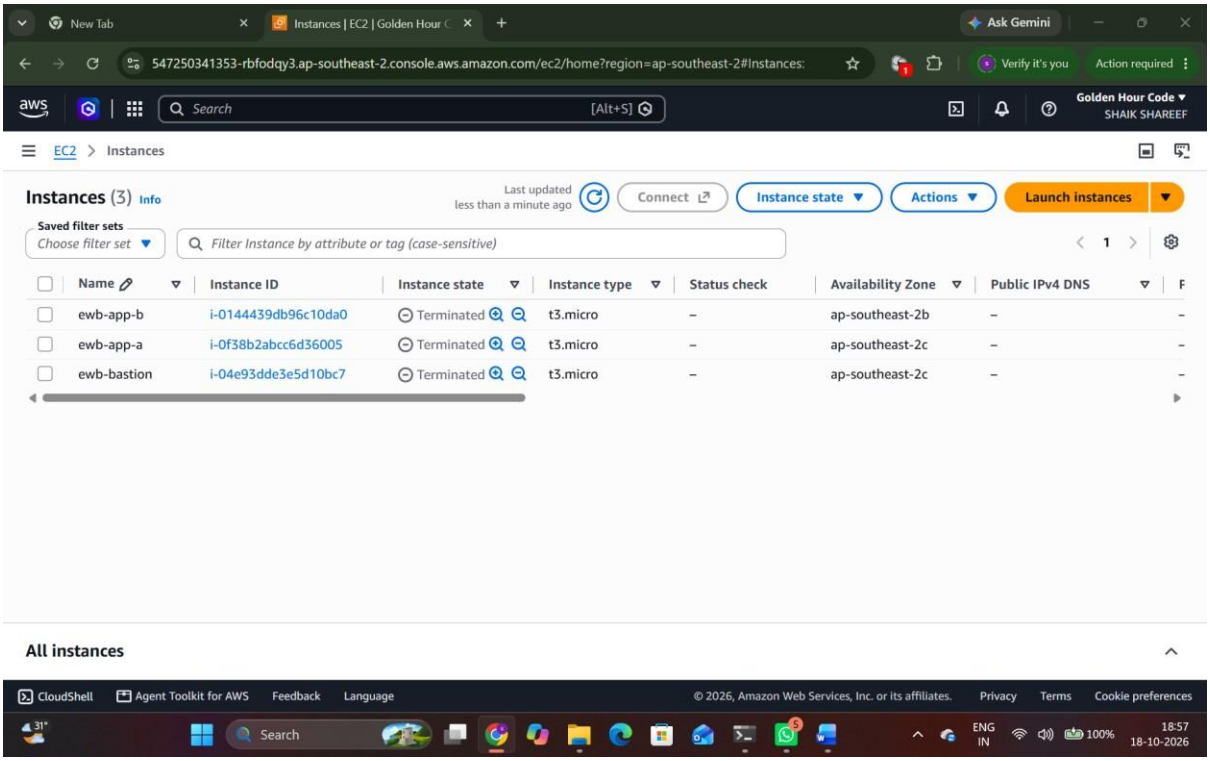


Fig 28: Terminated the all instances

11.3 Final Conclusion

This project provided practical experience in AWS VPC networking, peering connections, route table configuration, security groups, and private EC2 access through a bastion host. It improved my understanding of secure communication between two VPCs.